

Lab 1 – R Tutorial - Extra material - First Part

R – VECTORS

Vectors are the most basic R data objects and there are six types of atomic vectors. They are logical, integer, double, complex, character and raw.

Vector Creation

Single Element Vector

Even when you write just one value in R, it becomes a vector of length 1 and belongs to one of the above vector types.

```
# Atomic vector of type character.
```

```
print("abc");
```

```
# Atomic vector of type double.
```

```
print(12.5)
```

```
# Atomic vector of type integer.
```

```
print(63L)
```

```
# Atomic vector of type logical.
```

```
print(TRUE)
```

```
# Atomic vector of type complex.
```

```
print(2+3i)
```

```
# Atomic vector of type raw.
```

```
print(charToRaw('hello'))
```

When we execute the above code, it produces the following result:

```
[1] "abc"
```

```
[1] 12.5
[1] 63
[1] TRUE
[1] 2+3i
[1] 68 65 6c 6c 6f
```

Multiple Elements Vector

Using colon operator with numeric data

```
# Creating a sequence from 5 to 13.
```

```
v <- 5:13
```

```
print(v)
```

```
# Creating a sequence from 6.6 to 12.6.
```

```
v <- 6.6:12.6
```

```
print(v)
```

```
# If the final element specified does not belong to the sequence
then it is discarded.
```

```
v <- 3.8:11.4
```

```
print(v)
```

When we execute the above code, it produces the following result:

```
[1] 5 6 7 8 9 10 11 12 13
[1] 6.6 7.6 8.6 9.6 10.6 11.6 12.6
[1] 3.8 4.8 5.8 6.8 7.8 8.8 9.8 10.8
```

Using sequence (Seq.) operator

```
# Create vector with elements from 5 to 9 incrementing by 0.4.
```

```
print(seq(5, 9, by = 0.4))
```

When we execute the above code, it produces the following result:

```
[1] 5.0 5.4 5.8 6.2 6.6 7.0 7.4 7.8 8.2 8.6 9.0
```

Using the c() function

The non-character values are coerced to character type if one of the elements is a character.

```
# The logical and numeric values are converted to characters.
s <- c('apple','red',5,TRUE)
print(s)
```

When we execute the above code, it produces the following result:

```
[1] "apple" "red"    "5"      "TRUE"
```

Accessing Vector Elements

Elements of a Vector are accessed using indexing. The **[]** brackets are used for indexing. Indexing starts with position 1. Giving a negative value in the index drops that element from result. **TRUE**, **FALSE** or **0** and **1** can also be used for indexing.

```
# Accessing vector elements using position.
t <- c("Sun", "Mon", "Tue", "Wed", "Thurs", "Fri", "Sat")
u <- t[c(2,3,6)]
print(u)
```

```
# Accessing vector elements using logical indexing.
v <- t[c(TRUE,FALSE,FALSE,FALSE,FALSE,TRUE,FALSE)]
print(v)
```

```
# Accessing vector elements using negative indexing.
```

```

x <- t[c(-2,-5)]
print(x)

# Accessing vector elements using 0/1 indexing.
y <- t[c(0,0,0,0,0,0,1)]
print(y)

```

When we execute the above code, it produces the following result:

```

[1] "Mon" "Tue" "Fri"
[1] "Sun" "Fri"
[1] "Sun" "Tue" "Wed" "Fri" "Sat"
[1] "Sun"

```

Vector Manipulation

Vector arithmetic

Two vectors of same length can be added, subtracted, multiplied or divided giving the result as a vector output.

```

# Create two vectors.
v1 <- c(3,8,4,5,0,11)
v2 <- c(4,11,0,8,1,2)

# Vector addition.
add.result <- v1+v2
print(add.result)

# Vector subtraction.
sub.result <- v1-v2
print(sub.result)

```

```
# Vector multiplication.
multi.result <- v1*v2
print(multi.result)
```

```
# Vector division.
divi.result <- v1/v2
print(divi.result)
```

When we execute the above code, it produces the following result:

```
[1]  7 19  4 13  1 13
[1] -1 -3  4 -3 -1  9
[1] 12 88  0 40  0 22
[1] 0.7500000 0.7272727      Inf 0.6250000 0.0000000 5.5000000
```

Vector Element Recycling

If we apply arithmetic operations to two vectors of unequal length, then the elements of the shorter vector are recycled to complete the operations.

```
v1 <- c(3,8,4,5,0,11)
v2 <- c(4,11)
# V2 becomes c(4,11,4,11,4,11)
```

```
add.result <- v1+v2
print(add.result)
```

```
sub.result <- v1-v2
print(sub.result)
```

When we execute the above code, it produces the following result:

```
[1]  7 19  8 16  4 22
```

```
[1] -1 -3 0 -6 -4 0
```

Vector Element Sorting

Elements in a vector can be sorted using the **sort()** function.

```
v <- c(3,8,4,5,0,11, -9, 304)
```

```
# Sort the elements of the vector.
```

```
sort.result <- sort(v)
```

```
print(sort.result)
```

```
# Sort the elements in the reverse order.
```

```
revsort.result <- sort(v, decreasing = TRUE)
```

```
print(revsort.result)
```

```
# Sorting character vectors.
```

```
v <- c("Red","Blue","yellow","violet")
```

```
sort.result <- sort(v)
```

```
print(sort.result)
```

```
# Sorting character vectors in reverse order.
```

```
revsort.result <- sort(v, decreasing = TRUE)
```

```
print(revsort.result)
```

When we execute the above code, it produces the following result:

```
[1] -9 0 3 4 5 8 11 304
```

```
[1] 304 11 8 5 4 3 0 -9
```

```
[1] "Blue" "Red" "violet" "yellow"
```

```
[1] "yellow" "violet" "Red" "Blue"
```

R – LISTS

Lists are the R objects which contain elements of different types like – numbers, strings, vectors and another list inside it. A list can also contain a matrix or a function as its elements. List is created using **list()** function.

Creating a List

Following is an example to create a list containing strings, numbers, vectors and logical values.

```
# Create a list containing strings, numbers, vectors and a logical
# values.
list_data <- list("Red", "Green", c(21,32,11), TRUE, 51.23, 119.1)
print(list_data)
```

When we execute the above code, it produces the following result:

```
[[1]]
[1] "Red"

[[2]]
[1] "Green"

[[3]]
[1] 21 32 11

[[4]]
[1] TRUE

[[5]]
[1] 51.23
```

```
[[6]]  
[1] 119.1
```

Naming List Elements

The list elements can be given names and they can be accessed using these names.

```
# Create a list containing a vector, a matrix and a list.  
list_data <- list(c("Jan","Feb","Mar"), matrix(c(3,9,5,1,-2,8),  
nrow = 2),  
  list("green",12.3))  
  
# Give names to the elements in the list.  
names(list_data) <- c("1st Quarter", "A_Matrix", "A Inner list")  
  
# Show the list.  
print(list_data)
```

When we execute the above code, it produces the following result:

```
$`1st_Quarter`  
[1] "Jan" "Feb" "Mar"  
  
$A_Matrix  
      [,1] [,2] [,3]  
[1,]    3    5  -2  
[2,]    9    1    8  
  
$A_Inner_list  
$A_Inner_list[[1]]  
[1] "green"
```



```
$A_Inner_list[[2]]  
[1] 12.3
```

Accessing List Elements

Elements of the list can be accessed by the index of the element in the list. In case of named lists it can also be accessed using the names.

We continue to use the list in the above example.

```
# Create a list containing a vector, a matrix and a list.  
list_data <- list(c("Jan","Feb","Mar"), matrix(c(3,9,5,1,-2,8),  
nrow = 2),  
  list("green",12.3))  
  
# Give names to the elements in the list.  
names(list_data) <- c("1st Quarter", "A_Matrix", "A Inner list")  
  
# Access the first element of the list.  
print(list_data[1])  
  
# Access the thrid element. As it is also a list, all its elements  
will be printed.  
print(list_data[3])  
  
# Access the list element using the name of the element.  
print(list_data$A_Matrix)
```

When we execute the above code, it produces the following result:

```
$`1st_Quarter`  
[1] "Jan" "Feb" "Mar"  
  
$A_Inner_list
```

```
$A_Inner_list[[1]]  
[1] "green"
```

```
$A_Inner_list[[2]]  
[1] 12.3
```

```
      [,1] [,2] [,3]  
[1,]     3     5    -2  
[2,]     9     1     8
```

Manipulating List Elements

We can add, delete and update list elements as shown below. We can add and delete elements only at the end of a list. But we can update any element.

```
# Create a list containing a vector, a matrix and a list.  
list_data <- list(c("Jan","Feb","Mar"), matrix(c(3,9,5,1,-2,8),  
nrow = 2),  
  list("green",12.3))  
  
# Give names to the elements in the list.  
names(list_data) <- c("1st Quarter", "A_Matrix", "A Inner list")  
  
# Add element at the end of the list.  
list_data[4] <- "New element"  
print(list_data[4])  
  
# Remove the last element.  
list_data[4] <- NULL  
  
# Print the 4th Element.  
print(list_data[4])
```

```
# Update the 3rd Element.  
list_data[3] <- "updated element"  
print(list_data[3])
```

When we execute the above code, it produces the following result:

```
[[1]]  
[1] "New element"
```

```
$<NA>  
NULL
```

```
$`A Inner list`  
[1] "updated element"
```

Merging Lists

You can merge many lists into one list by placing all the lists inside one **list()** function.

```
# Create two lists.  
list1 <- list(1,2,3)  
list2 <- list("Sun", "Mon", "Tue")  
  
# Merge the two lists.  
merged.list <- c(list1,list2)  
  
# Print the merged list.  
print(merged.list)
```

When we execute the above code, it produces the following result:

```
[[1]]  
[1] 1
```

```
[[2]]
```

```
[1] 2
```

```
[[3]]
```

```
[1] 3
```

```
[[4]]
```

```
[1] "Sun"
```

```
[[5]]
```

```
[1] "Mon"
```

```
[[6]]
```

```
[1] "Tue"
```

Converting List to Vector

A list can be converted to a vector so that the elements of the vector can be used for further manipulation. All the arithmetic operations on vectors can be applied after the list is converted into vectors. To do this conversion, we use the `unlist()` function. It takes the list as input and produces a vector.

```
# Create lists.
```

```
list1 <- list(1:5)
```

```
print(list1)
```

```
list2 <-list(10:14)
```

```
print(list2)
```

```
# Convert the lists to vectors.
```

```
v1 <- unlist(list1)
```

```
v2 <- unlist(list2)
```

```
print(v1)
```

```
print(v2)
```

```
# Now add the vectors
```

```
result <- v1+v2
```

```
print(result)
```

When we execute the above code, it produces the following result:

```
[[1]]
```

```
[1] 1 2 3 4 5
```

```
[[1]]
```

```
[1] 10 11 12 13 14
```

```
[1] 1 2 3 4 5
```

```
[1] 10 11 12 13 14
```

```
[1] 11 13 15 17 19
```

R – MATRICES

Matrices are the R objects in which the elements are arranged in a two-dimensional rectangular layout. They contain elements of the same atomic types. Though we can create a matrix containing only characters or only logical values, they are not of much use. We use matrices containing numeric elements to be used in mathematical calculations.

A Matrix is created using the **matrix()** function.

Syntax

The basic syntax for creating a matrix in R is –

```
matrix(data, nrow, ncol, byrow, dimnames)
```

Following is the description of the parameters used:

- **data** is the input vector which becomes the data elements of the matrix.
- **nrow** is the number of rows to be created.
- **ncol** is the number of columns to be created.
- **byrow** is a logical clue. If **TRUE** then the input vector elements are arranged by row.
- **dimname** is the names assigned to the rows and columns.

Example

Create a matrix taking a vector of numbers as input.

```
# Elements are arranged sequentially by row.  
M <- matrix(c(3:14), nrow = 4, byrow = TRUE)  
print(M)
```

```
# Elements are arranged sequentially by column.  
N <- matrix(c(3:14), nrow = 4, byrow = FALSE)  
print(N)
```

```
# Define the column and row names.

rownames = c("row1", "row2", "row3", "row4")
colnames = c("col1", "col2", "col3")

P <- matrix(c(3:14), nrow = 4, byrow = TRUE, dimnames =
list(rownames, colnames))

print(P)
```

When we execute the above code, it produces the following result:

```
      [,1] [,2] [,3]
[1,]     3     4     5
[2,]     6     7     8
[3,]     9    10    11
[4,]    12    13    14

      [,1] [,2] [,3]
[1,]     3     7    11
[2,]     4     8    12
[3,]     5     9    13
[4,]     6    10    14

      col1 col2 col3
row1     3     4     5
row2     6     7     8
row3     9    10    11
row4    12    13    14
```

Accessing Elements of a Matrix

Elements of a matrix can be accessed by using the column and row index of the element. We consider the matrix P above to find the specific elements below.

```

# Define the column and row names.
rownames = c("row1", "row2", "row3", "row4")
colnames = c("col1", "col2", "col3")

# Create the matrix.
P <- matrix(c(3:14), nrow = 4, byrow = TRUE, dimnames =
list(rownames, colnames))

# Access the element at 3rd column and 1st row.
print(P[1,3])

# Access the element at 2nd column and 4th row.
print(P[4,2])

# Access only the 2nd row.
print(P[2,])

# Access only the 3rd column.
print(P[,3])

```

When we execute the above code, it produces the following result:

```

[1] 5
[1] 13
col1 col2 col3
    6    7    8
row1 row2 row3 row4
    5    8   11   14

```


Matrix Computations

Various mathematical operations are performed on the matrices using the R operators. The result of the operation is also a matrix.

The dimensions (number of rows and columns) should be same for the matrices involved in the operation.

Matrix Addition & Subtraction

```
# Create two 2x3 matrices.
matrix1 <- matrix(c(3, 9, -1, 4, 2, 6), nrow = 2)
print(matrix1)

matrix2 <- matrix(c(5, 2, 0, 9, 3, 4), nrow = 2)
print(matrix2)

# Add the matrices.
result <- matrix1 + matrix2
cat("Result of addition","\n")
print(result)

# Subtract the matrices
result <- matrix1 - matrix2
cat("Result of subtraction","\n")
print(result)
```

When we execute the above code, it produces the following result:

```
      [,1] [,2] [,3]
[1,]    3  -1    2
[2,]    9   4    6
```

	[,1]	[,2]	[,3]
[1,]	5	0	3
[2,]	2	9	4

Result of addition

	[,1]	[,2]	[,3]
[1,]	8	-1	5
[2,]	11	13	10

Result of subtraction

	[,1]	[,2]	[,3]
[1,]	-2	-1	-1
[2,]	7	-5	2

Matrix Multiplication & Division

Create two 2x3 matrices.

```
matrix1 <- matrix(c(3, 9, -1, 4, 2, 6), nrow = 2)
print(matrix1)
```

```
matrix2 <- matrix(c(5, 2, 0, 9, 3, 4), nrow = 2)
print(matrix2)
```

Multiply the matrices.

```
result <- matrix1 * matrix2
cat("Result of multiplication","\n")
print(result)
```

Divide the matrices

```
result <- matrix1 / matrix2
cat("Result of division","\n")
print(result)
```

When we execute the above code, it produces the following result:

```
      [,1] [,2] [,3]
[1,]    3  -1    2
[2,]    9   4    6
      [,1] [,2] [,3]
[1,]    5   0    3
[2,]    2   9    4
Result of multiplication
      [,1] [,2] [,3]
[1,]   15   0   6
[2,]   18  36  24
Result of division
      [,1]      [,2]      [,3]
[1,]  0.6      -Inf  0.6666667
[2,]  4.5  0.4444444  1.5000000
```

R – ARRAYS

Arrays are the R data objects which can store data in more than two dimensions. For example – If we create an array of dimension (2, 3, 4) then it creates 4 rectangular matrices each with 2 rows and 3 columns. Arrays can store only data type.

An array is created using the **array()** function. It takes vectors as input and uses the values in the dim parameter to create an array.

Example

The following example creates an array of two 3x3 matrices each with 3 rows and 3 columns.

```
# Create two vectors of different lengths.
vector1 <- c(5,9,3)
vector2 <- c(10,11,12,13,14,15)

# Take these vectors as input to the array.
result <- array(c(vector1,vector2),dim = c(3,3,2))
print(result)
```

When we execute the above code, it produces the following result:

```
, , 1
```

```
      [,1] [,2] [,3]
[1,]    5   10   13
[2,]    9   11   14
[3,]    3   12   15
```

```
, , 2
```

	[,1]	[,2]	[,3]
[1,]	5	10	13
[2,]	9	11	14
[3,]	3	12	15

Naming Columns and Rows

We can give names to the rows, columns and matrices in the array by using the **dimnames** parameter.

```
# Create two vectors of different lengths.
vector1 <- c(5,9,3)
vector2 <- c(10,11,12,13,14,15)
column.names <- c("COL1","COL2","COL3")
row.names <- c("ROW1","ROW2","ROW3")
matrix.names <- c("Matrix1","Matrix2")

# Take these vectors as input to the array.
result <- array(c(vector1,vector2),dim = c(3,3,2),dimnames =
list(row.names,column.names,
      matrix.names))
print(result)
```

When we execute the above code, it produces the following result:

```
, , Matrix1

      COL1 COL2 COL3
ROW1    5   10   13
ROW2    9   11   14
ROW3    3   12   15
```

```
, , Matrix2
```

	COL1	COL2	COL3
ROW1	5	10	13
ROW2	9	11	14
ROW3	3	12	15

Accessing Array Elements

```
# Create two vectors of different lengths.
vector1 <- c(5,9,3)
vector2 <- c(10,11,12,13,14,15)
column.names <- c("COL1","COL2","COL3")
row.names <- c("ROW1","ROW2","ROW3")
matrix.names <- c("Matrix1","Matrix2")

# Take these vectors as input to the array.
result <- array(c(vector1,vector2),dim = c(3,3,2),dimnames =
  list(row.names,
    column.names, matrix.names))

# Print the third row of the second matrix of the array.
print(result[3,,2])

# Print the element in the 1st row and 3rd column of the 1st matrix.
print(result[1,3,1])

# Print the 2nd Matrix.
print(result[, ,2])
```

When we execute the above code, it produces the following result:

```
COL1 COL2 COL3
    3   12   15
[1] 13
      COL1 COL2 COL3
ROW1     5   10   13
ROW2     9   11   14
ROW3     3   12   15
```

Manipulating Array Elements

As array is made up matrices in multiple dimensions, the operations on elements of array are carried out by accessing elements of the matrices.

```
# Create two vectors of different lengths.
```

```
vector1 <- c(5,9,3)
```

```
vector2 <- c(10,11,12,13,14,15)
```

```
# Take these vectors as input to the array.
```

```
array1 <- array(c(vector1,vector2),dim = c(3,3,2))
```

```
# Create two vectors of different lengths.
```

```
vector3 <- c(9,1,0)
```

```
vector4 <- c(6,0,11,3,14,1,2,6,9)
```

```
array2 <- array(c(vector1,vector2),dim = c(3,3,2))
```

```
# create matrices from these arrays.
```

```
matrix1 <- array1[, ,2]
```

```
matrix2 <- array2[, ,2]
```

```
# Add the matrices.  
result <- matrix1+matrix2  
print(result)
```

When we execute the above code, it produces the following result:

```
      [,1] [,2] [,3]  
[1,]   10   20   26  
[2,]   18   22   28  
[3,]    6   24   30
```

Calculations Across Array Elements

We can do calculations across the elements in an array using the **apply()** function.

Syntax

```
apply(x, margin, fun)
```

Following is the description of the parameters used:

- **x** is an array.
- **margin** is the name of the data set used.
- **fun** is the function to be applied across the elements of the array.

Example

We use the **apply()** function below to calculate the sum of the elements in the rows of an array across all the matrices.

```
# Create two vectors of different lengths.  
vector1 <- c(5,9,3)
```



```

vector2 <- c(10,11,12,13,14,15)

# Take these vectors as input to the array.
new.array <- array(c(vector1,vector2),dim = c(3,3,2))
print(new.array)

# Use apply to calculate the sum of the rows across all the matrices.
result <- apply(new.array, c(1), sum)
print(result)

```

When we execute the above code, it produces the following result:

```
, , 1
```

	[,1]	[,2]	[,3]
[1,]	5	10	13
[2,]	9	11	14
[3,]	3	12	15

```
, , 2
```

	[,1]	[,2]	[,3]
[1,]	5	10	13
[2,]	9	11	14
[3,]	3	12	15

```
[1] 56 68 60
```

R – FACTORS

Factors are the data objects which are used to categorize the data and store it as levels. They can store both strings and integers. They are useful in the columns which have a limited number of unique values. Like "Male", "Female" and True, False etc. They are useful in data analysis for statistical modelling.

Factors are created using the **factor ()** function by taking a vector as input.

Example

```
# Create a vector as input.

data <- c("East", "West", "East", "North", "North", "East", "West", "West", "West",
          "East", "North")

print(data)

print(is.factor(data))

# Apply the factor function.

factor_data <- factor(data)

print(factor_data)

print(is.factor(factor_data))
```

When we execute the above code, it produces the following result:

```
[1] "East"  "West"  "East"  "North" "North" "East"  "West"  "West"
"West"  "East"  "North"

[1] FALSE

[1] East  West  East  North North East  West  West  West  East  North
Levels: East North West

[1] TRUE
```

Factors in Data Frame

On creating any data frame with a column of text data, R treats the text column as categorical data and creates factors on it.

```
# Create the vectors for data frame.
height <- c(132,151,162,139,166,147,122)
weight <- c(48,49,66,53,67,52,40)
gender <- c("male","male","female","female","male","female","male")

# Create the data frame.
input_data <- data.frame(height,weight,gender)
print(input_data)

# Test if the gender column is a factor.
print(is.factor(input_data$gender))

# Print the gender column so see the levels.
print(input_data$gender)
```

When we execute the above code, it produces the following result:

```
  height weight gender
1    132    48   male
2    151    49   male
3    162    66 female
4    139    53 female
5    166    67   male
6    147    52 female
7    122    40   male

[1] TRUE

[1] male   male   female female male   female male
```

```
Levels: female male
```

Changing the Order of Levels

The order of the levels in a factor can be changed by applying the factor function again with new order of the levels.

```
data <- c("East","West","East","North","North","East","West",
         "West","West","East","North")

# Create the factors
factor_data <- factor(data)
print(factor_data)

# Apply the factor function with required order of the level.
new_order_data <- factor(factor_data,levels =
c("East","West","North"))
print(new_order_data)
```

When we execute the above code, it produces the following result:

```
[1] East  West  East  North North East  West  West  West  East
North
Levels: East North West

[1] East  West  East  North North East  West  West  West  East
North
Levels: East West North
```

Generating Factor Levels

We can generate factor levels by using the `gl()` function. It takes two integers as input which indicates how many levels and how many times each level.

Syntax

```
gl(n, k, labels)
```

Following is the description of the parameters used –

- **n** is an integer giving the number of levels.
- **k** is an integer giving the number of replications.
- **labels** is a vector of labels for the resulting factor levels.

Example

```
v <- gl(3, 4, labels = c("Tampa", "Seattle", "Boston"))  
print(v)
```

When we execute the above code, it produces the following result:

```
Tampa   Tampa   Tampa   Tampa   Seattle Seattle Seattle Seattle  
Boston  
  
[10] Boston Boston Boston  
  
Levels: Tampa Seattle Boston
```